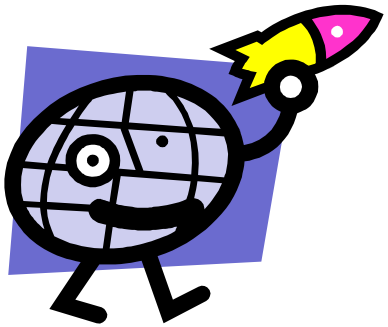
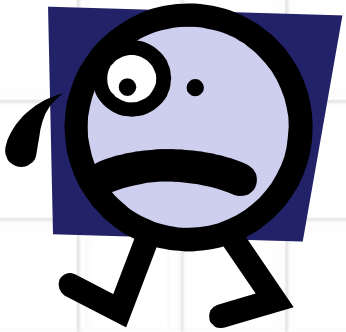




LAB

Software Engineering



งานที่ 1 : การสร้างหน้าเพจหลัก index



ครั้งที่ 1 :

พัฒนาตาม UI-ต้นแบบ ที่มีคุณสมบัติดังนี้

- สร้าง index.html
- สร้าง style.css พร้อม comment
- เขียน css แบบ external
- ต้องกำหนด สีธีมของผลงานตนเอง (ห้ามซ้ำ)
- ต้องแสดงผลลัพธ์ตาม UI

งานที่ 1 : การสร้างหน้าเพจหลัก index

สรุปการจำแนกงาน (Architecture Breakdown)

- **index.html** ทำหน้าที่สร้างโครงสร้างหลัก (DOM) แบ่งออกเป็น 4 ส่วน ได้แก่
  - แถบเมนูด้านบน
  - ป้ายสไลด์ประชาสัมพันธ์ (*ตอนนี้ยังไม่สไลด์*)
  - การ์ดสินค้าขายดี/สินค้าแนะนำ (*ตอนนี้จำลองสินค้าขายดี 5 รายการ*)
  - แถบค้นหาพร้อมตัวกรองประเภทสินค้า และตารางแสดงสินค้าแบบ Grid Layout (*ตอนนี้จำลองสินค้าทั้งหมด 6 รายการ*)
- **style.css** จัดการธีมสีให้ตรงตามรูปภาพ และใช้ CSS Grid

งานที่ 1 : การสร้างหน้าเพจหลัก index.html



- ○แถบเมนูด้านบน
- ○ป้ายสไลด์ประชาสัมพันธ์
- ○การ์ดสินค้าขายดี/สินค้าแนะนำ
  (*จำลองสินค้าขายดี 5 รายการ*)
- ○แถบค้นหาพร้อมตัวกรองประเภทสินค้า
  และตารางแสดงสินค้า
  (*จำลองสินค้าทั้งหมด 6 รายการ*)

งานที่ 1 : การสร้างหน้าเพจหลัก index แบบ Responsive (Media Queries)



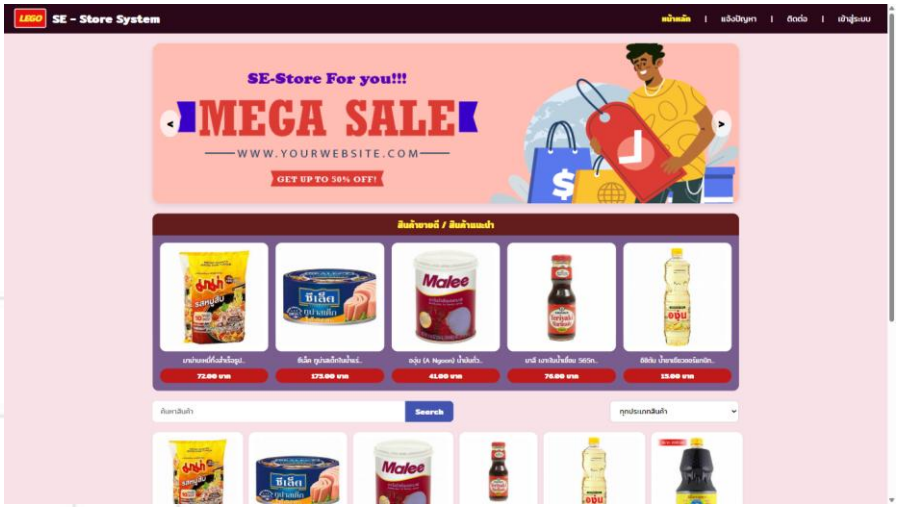
ครั้งที่ 2 :

พัฒนาเพิ่มเติม มีคุณสมบัติดังนี้

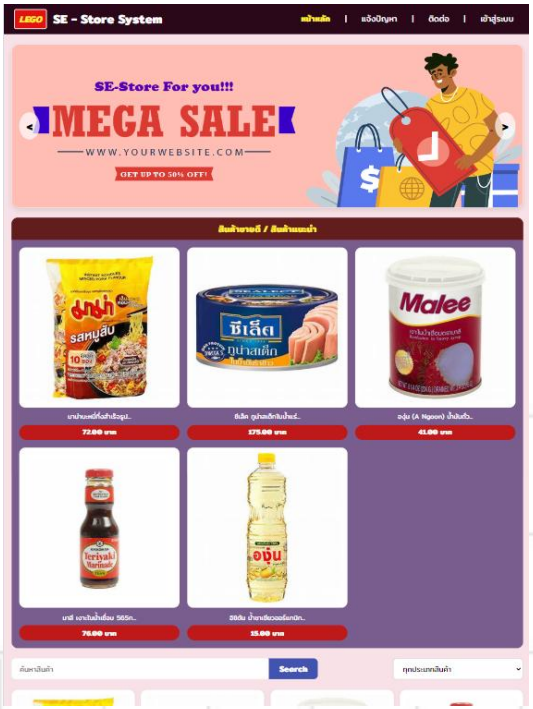
- ออกแบบเมนูแนวนอนพร้อม hover effect แบบเปลี่ยนสี ขยายขนาด
- กำหนดให้รองรับหลายอุปกรณ์ และเปลี่ยน layout และตัวอักษร
- ใช้ Media Queries ในการซ่อน/แสดงเมนูที่เหมาะสมกับขนาดหน้าจอ (Hamburger Menu)
- ใช้ Pseudo-class พร้อมกับ Transition และ Transform
- สร้างลิงค์เรียกใช้เพจ login.html

งานที่ 1 : การสร้างหน้าเพจหลัก index แบบ Responsive

Desktop (default)



Tablet (<1080px)



Mobile (<576px)



งานที่ 1 : การสร้างหน้าเพจหลัก index : Fetch Data



ครั้งที่ 3 :

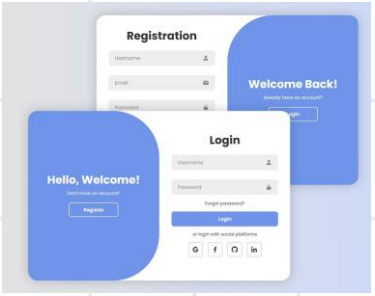
พัฒนาเพิ่มเติม มีคุณสมบัติดังนี้

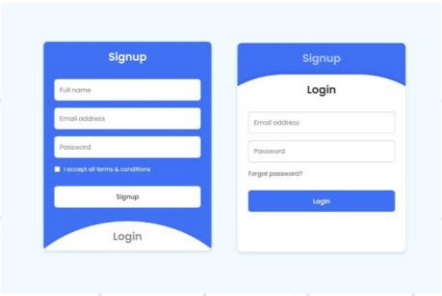
- สร้างภาพประชาสัมพันธ์แบบสไลด์ ไม่ต่ำกว่า 3 ภาพ
- การกำหนดติดต่อฐานข้อมูลแบบ Fetch..API
- การสร้าง Action ต่าง ๆ อาทิ การค้นหา หรือเลือกประเภท
- การเชื่อมโยงเพจ Login

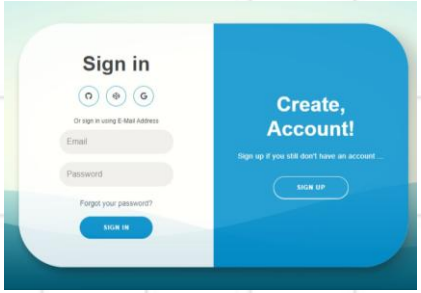
งานที่ 2 : การสร้างหน้าเพจ login : Front-end

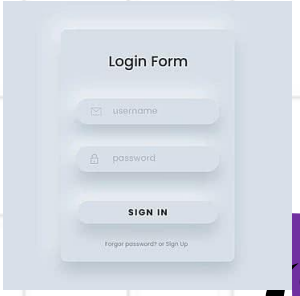
ครั้งที่ 1 : พัฒนาตาม UI-ต้นแบบ ที่มีคุณสมบัติดังนี้

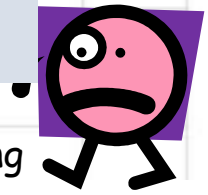
- สร้างไฟล์ **login.html** และ **login.css**
- กำหนดให้มีการเชื่อมโยงไปยังการลงทะเบียนผู้ใช้ใหม่
- เขียน CSS แบบ external
- ออกแบบ UI และแสดงผลลัพธ์ตามที่ออกแบบ โดยอยู่ในธีมที่กำหนด
- **<u>กำหนดการตอบสนองหลากหลาย</u>**เช่น เมื่อเมาส์อยู่เหนืออ๊อบเจ็คหนึ่ง ๆ (ค้นหาและศึกษารูปแบบใหม่ ๆ คนละแบบ)







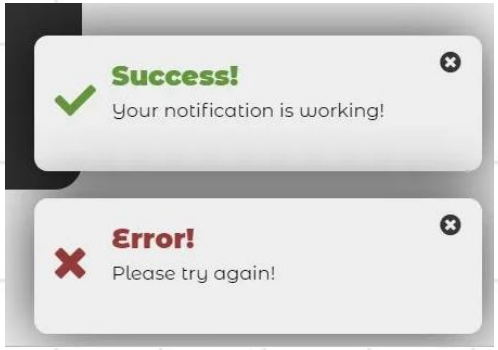


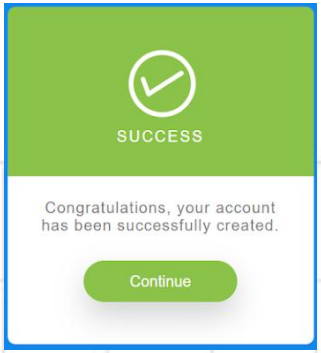


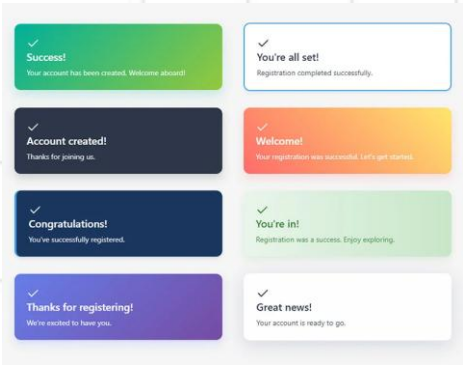
งานที่ 2 : การสร้างหน้าเพจ login : Back-end

ครั้งที่ 2 : สร้าง Backend สำหรับตรวจสอบสิทธิ์ กับงาน Login มีคุณสมบัติดังนี้

- ❑ สร้างไฟล์ตรวจสอบความถูกต้องของข้อมูลก่อนยืนยัน **login.js**
- ❑ เชื่อมต่อฐานข้อมูล (สร้าง View)
- ❑ ตรวจสอบสิทธิ์ของผู้ใช้ ผ่าน REST API
- ❑ แสดงผลเมื่อการลงทะเบียนหรือเข้าสู่ระบบสำเร็จ/ไม่สำเร็จ
- ❑ เมื่อ Login สำเร็จ ให้แสดงข้อมูลรูปแบบ JSON



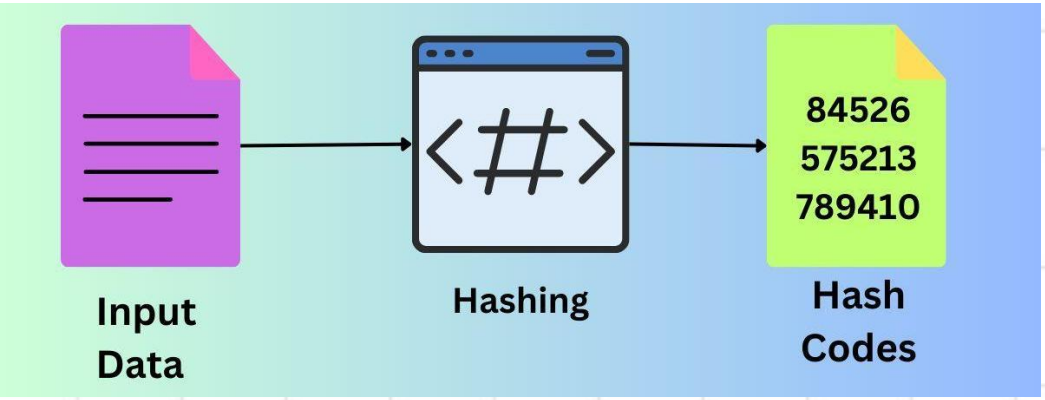




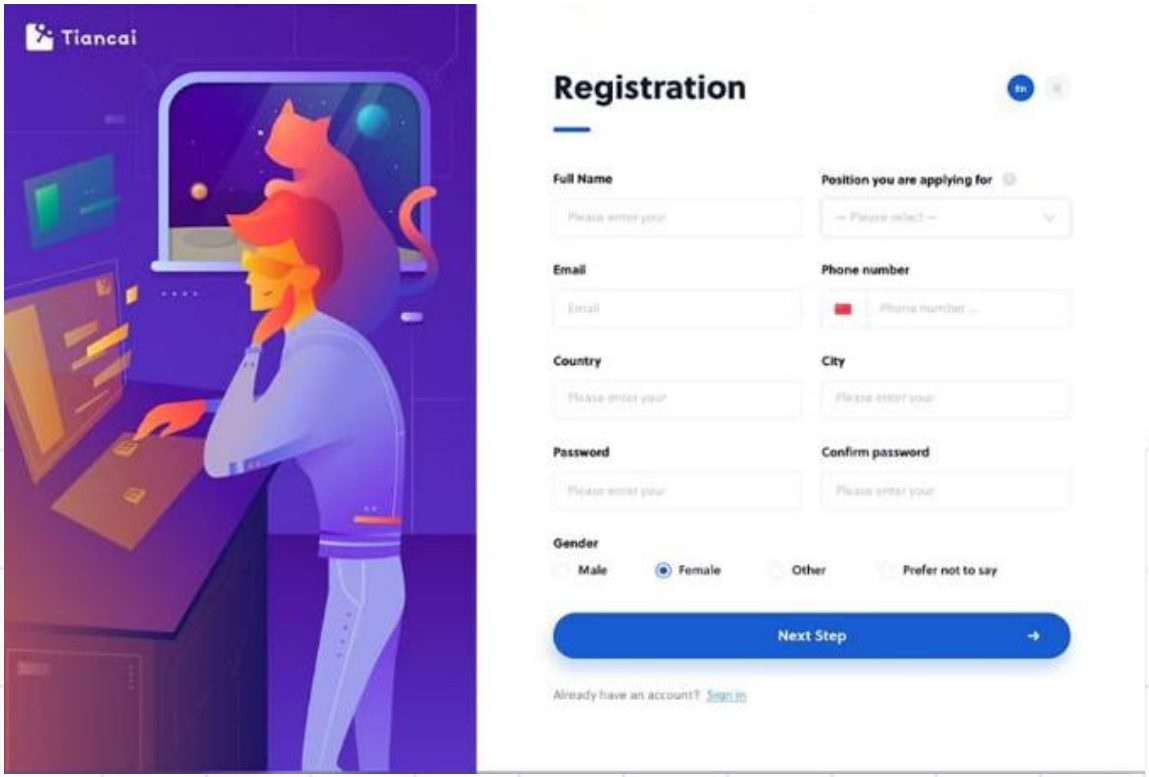
งานที่ 2 : การสร้างหน้าเพจ login : Authentication

ครั้งที่ 3 : เพิ่มการกระบวนการ Authentication ของ Login

- ตรวจสอบ Authentication เมื่อผู้ใช้ Login
- สร้าง Session Storage, Local Storage หรือ ตรวจสอบ Token/JWT สำหรับผู้ใช้งานที่เข้าสู่ระบบสำเร็จ



งานที่ 3 : การสร้างหน้าเพจ Register : Front-end

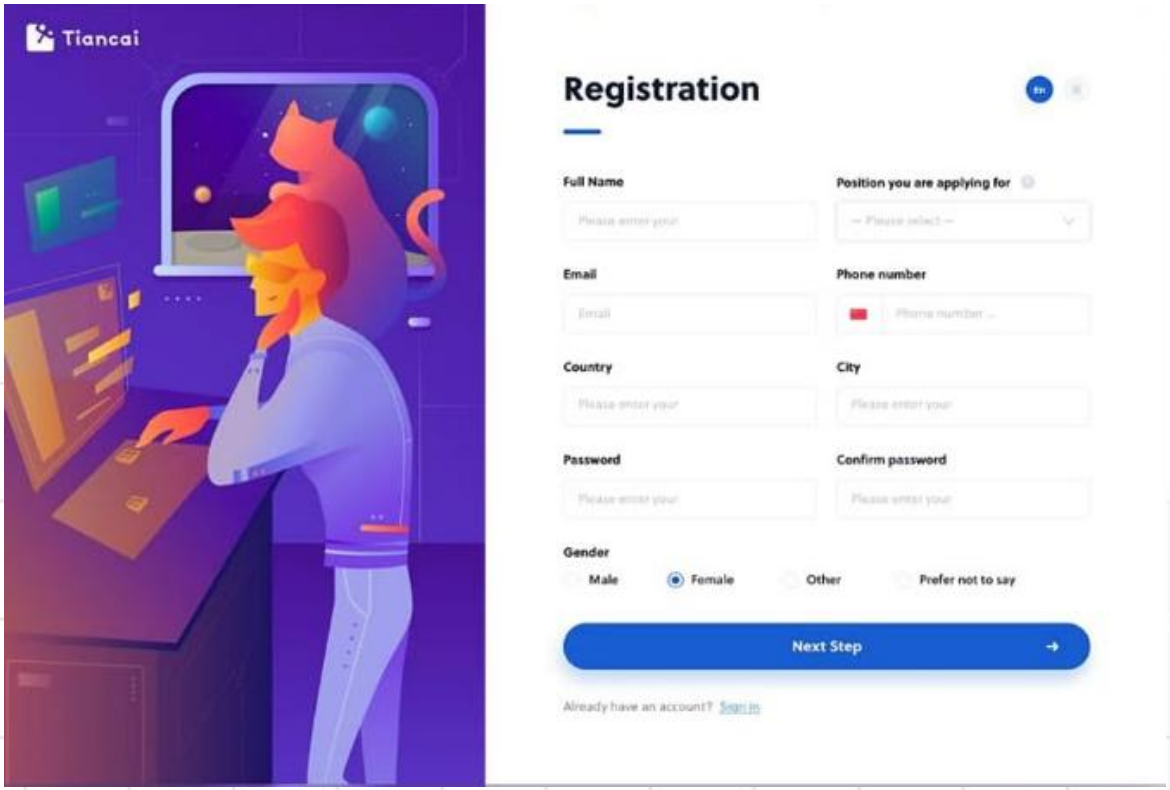


ครั้งที่ 1 :

พัฒนาตาม UI-ต้นแบบ ที่มีคุณสมบัติดังนี้

- สร้าง **register.html**
- สร้าง **register.css** พร้อม comment
- สร้าง **register.js** ตรวจสอบการทุกการกรอกข้อมูลตามรูปแบบที่กำหนด
- โต้ตอบกับหน้าเว็บด้วย DOM

งานที่ 3 : การสร้างหน้าเพจ Register : Back-end

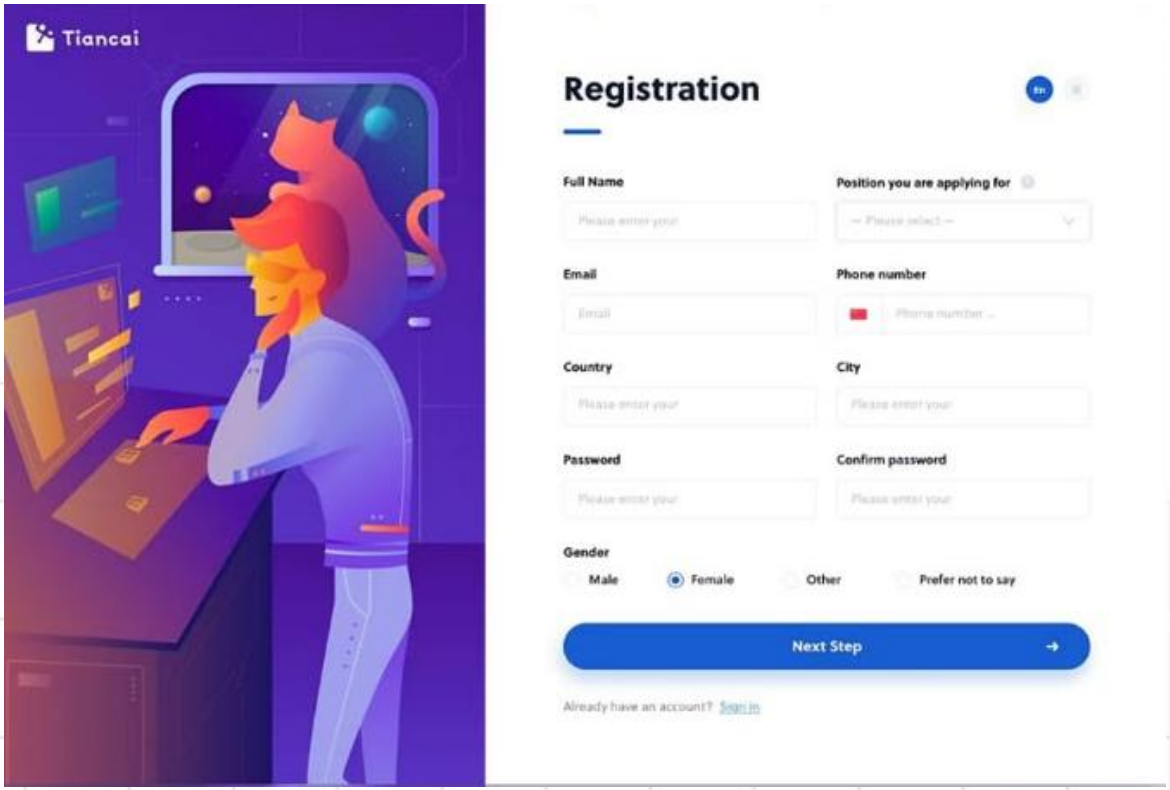


ครั้งที่ 2 :

สร้าง Backend สำหรับตรวจจัดเก็บข้อมูลผู้ลงทะเบียนกับงาน Register มีคุณสมบัติดังนี้

- เชื่อมต่อฐานข้อมูล
- ตรวจสอบข้อมูลรับข้อมูล
- ตรวจสอบข้อมูลซ้ำซ้อน
- บันทึกข้อมูลลงตารางผ่าน REST API
- เมื่อบันทึกสำเร็จ ให้แสดงข้อมูลรูปแบบ JSON

งานที่ 3 : การสร้างหน้าเพจ Register : Authentication



ครั้งที่ 3 :

การเพิ่ม Authenticationของ Register

- ปรับการจัดเก็บ Password ให้ใช้ Password Hashing

งานที่ 4 : การสร้างหน้าเพจ Dashboard : React



ครั้งที่ 1 :

พัฒนาตาม UI-ต้นแบบ ที่มีคุณสมบัติดังนี้

- สร้าง **Dashboard.js** แบบ React
- เชื่อมต่อเพจ Login หากตรวจสอบสิทธิ์ และเป็น Admin ให้เข้ามาแสดงเพจนี้

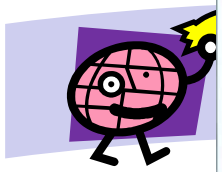
งานที่ 4 : การสร้างหน้าเพจ Dashboard : React



ครั้งที่ 2 :

การเพิ่มควบคุมการเข้าถึงของ ผู้ใช้ระบบ Dashboard

- กำหนดให้หน้า Dashboard สามารถเข้าถึงได้เฉพาะผู้ที่ผ่าน Authentication
- ป้องกันการเข้าถึงข้อมูลหรือ API ที่กำหนดจากผู้ที่ยังไม่ได้เข้าสู่ระบบ
- เพิ่มการทำงาน Logout



Welcome Back 👋

Login to your account

Email address

Password

Login

Don't have an account? Register

```html
<form id="authForm">
 <input id="email" type="email"
      placeholder="Email" required>
 <input id="password" type="password"
      placeholder="Password" required>
  <button type="submit">Login</button>
</form>
```

```js
const form = document.getElementById("authForm");

form.addEventListener("submit", async (e) => {
  e.preventDefault();

  const email = document.getElementById("email").value.trim();
  const password = document.getElementById("password").value;

  if (!email || password.length < 6) {
    alert("Enter valid email and 6+ character password");
    return;
  }

  // Call API for login
  const res = await fetch("/api/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password })
  });

  const data = await res.json();

  if (res.ok) {
    localStorage.setItem("token", data.token);
    window.location.href = "/dashboard.html";
  } else {
    alert(data.message);
  }
});
```

Valid
user@

Passv
Minim

API V
Check

Succ
Store

Error
Show

3 Token / Session Handling



- Store token in localStorage (or use HttpOnly cookies for more security)
- Send token with every protected API request
- Example using Fetch API

```js
const token = localStorage.getItem("token");

fetch("/api/profile", {
  headers: {
    Authorization: `Bearer ${token}`
  }
})
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Tip: For production apps, prefer HttpOnly + Secure cookies instead of localStorage to protect against XSS attacks.

4 Protected Routes



```js
const token = localStorage.getItem("token");

if (!token) {
  window.location.href = "/login.html";
}
```

This code checks if the user is authenticated. If not, it redirects to the login page.

5 Logout Functionality



```js
function logout() {
  localStorage.removeItem("token");
  window.location.href = "/login.html";
}
```

Lo

6 Authentication Flow



★ Secure • Scalable • User Friendly Authentication Flow

Best Practi

- Always validate or and server
- Use strong passw
- Use HttpOnly coo better security
- Implement rate lir
- Always use HTTPS

m-softtech

Code. Secure. Authenticate.

follow by m-soft

Modern Web Pr